

# sysxplore.com | BASH BRACKETS: (), {}, \$(), [], [[]]

## \$(commands)

Executes a command and captures its output. **Command substitution** allows the result of a command (in this case, `grep`) to be stored in a variable.

```
1 log_file="/var/log/syslog"
2 keyword="error"
3 output=$(grep "$keyword" "$log_file")
```

## { list; }

Executes a group of commands in the same **shell process**. Curly braces group commands together to be executed sequentially in the current environment.

```
1 { sudo apt install exa
2   echo exa
3   echo "Listed files using exa"; }
```

## (a b c )

Creates an array of values. Parentheses are used to define an **array**, allowing multiple elements to be stored in one variable.

```
1 files=(log.txt log2.txt log3.txt)
2 for file in "${files[@]}"; do
3     echo "Processing $file"
4 done
```

## ( list )

Executes a list of commands in a separate **subshell**. The commands inside the parentheses run in a child process, isolated from the main shell.

```
1 ( cd /home/user
2   ls
3   whoami )
```

## {range}

Expands to multiple strings. Brace expansion is a powerful way to generate sequences or multiple strings, useful for **batch operations**. The range can be numbers or characters.

```
1 for file in backup_{1..4}.tar.gz; do
2     mv $file /var/oldbackups
3 done
```

## \${expression}

Modifies variable content. **Parameter expansion** allows you to alter a variable's value, such as changing a file extension from `.txt` to `.bak`.

```
1 filename="report.txt"
2 backup_file="${filename%.txt}.bak"
3 echo "Backup file: $backup_file"
```

## \${variable}

Accesses a variable's value. This is another way to **reference** a variable, commonly used when you need to follow it with additional characters or text.

```
1 username="John"
2 greeting="Hello, ${username}!"
3 echo "$greeting"
```

## \$((expression))

Performs **arithmetic calculations**. The double parentheses are used for math operations, such as addition, multiplication, etc.

```
1 num1=5
2 num2=3
3 result=$((num1 * num2 + 1))
```

## [ expression ]

Tests a condition using single brackets. The `[ ]` denotes a **test command** that checks conditions, such as whether a file **exists**.

```
1 file="/etc/passwd"
2 if [ -f "$file" ]; then
3     echo "File exists"
4 fi
```

## [[ expression ]]

Tests a condition using double brackets. Double brackets are more flexible in Bash, supporting **advanced pattern** matching and logical operators.

```
1 user=$USER
2 if [[ $user = "root" ]]; then
3     echo "You are the root user"
4 fi
```